The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

14

Taking Payment for Orders

Part IV — Money and Identity

Taught in	Week 10
Laboratory	A sandbox gateway, and what happens when it times out
Assessment	**Project proposal**

This is a class extract. It is *Chapter 14* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 14

Taking Payment for Orders

Part 1 | Part 2 | Part 3 | **Part 4** | Part 5 | Part 6

Part IV — Money and Identity

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the handshake between a shop and a gateway, and **name** the three outcomes of a payment request.
- **Measure** a gateway's response-time distribution, and **choose** a timeout from it rather than from its median.
- **Explain** why a timed-out payment has an unknown outcome, and **design** what the shop does next.
- **Implement** idempotency, and **demonstrate** what happens without it.
- **State** what PCI DSS requires of a shop that never touches a card number, and **identify** what would change that.

Before You Start

Chapter 13: the parties, and the difference between authorisation, capture and settlement.
Chapter 10: the order state machine, status 0, and why the order identifier exists before the payment.

Key Terms in This Chapter

gateway integration • hosted page • direct post • server-to-server • callback • webhook • timeout • latency distribution • percentile • long tail • idempotency key • replay • reconciliation • PCI DSS • SAQ-A

14.1 Three Ways to Take a Card, and Why One of Them Wins

A shop that wants a card number has three architectures available, and the choice determines how much of the payment industry's compliance regime applies to it.

Hosted page, Direct post, Server-to-server

A **hosted page** sends the customer to the provider's own site to type their card details, and receives them back afterwards with a result. The card number never reaches your server.

A **direct post** keeps the customer on your page, but the form submits straight to the provider. Your HTML is involved; your server is not.

Server-to-server means the card number arrives at your server and you forward it. You now hold card data, and almost every obligation in this chapter multiplies.

For a shop of the size this course is about, the answer is always the first or the second, and the reason is in the next box.

What PCI DSS actually asks of you

The Payment Card Industry Data Security Standard governs anyone who stores, processes or transmits card data. Its reporting burden is graded by how much of that you do, through a Self-Assessment Questionnaire.

- **SAQ-A** is for a merchant who has *entirely outsourced* card handling to a validated third party — a hosted page. It is the shortest questionnaire, and most of it asks whether you are keeping your own site from interfering with the provider's.
- **SAQ-A-EP** applies to direct post: your page constructs the form, so your page can be tampered with, and you are asked more.
- **SAQ-D** is the full standard, and is what server-to-server costs you. It runs to hundreds of requirements.

The design decision and the compliance decision are the same decision. Choosing a hosted page is not a shortcut around PCI DSS; it is the mechanism the standard offers for staying out of scope. A student who understands only this from the chapter has understood the important part.

What would change it: storing the card number "for the customer's convenience", logging the full request body, or accepting the number on your page "just to validate it". Each of these moves a shop from SAQ-A to SAQ-D with one line of code and no announcement.

14.2 The Course Ships Its Own Gateway

Measuring a real gateway needs a merchant account, which needs a registered business. A sandbox needs credentials that cannot go in a handout and will have expired before anyone reads it. So the stack in Appendix A includes a provider that does not exist.

What `code/gateway/provider.py` is, and is not

It is not a toy card processor. It implements nothing about cards. It is a gateway reduced to the **three properties that make integrating with one difficult**, each configurable so that you can make it behave worse on purpose:

- **It is slow, and irregularly so.** A median around 300 ms, with a minority of requests taking ten times that.
- **It sometimes does not answer at all.** About one request in fifty hangs, which is the case the rest of the chapter is about.
- **It tells you twice.** It honours an idempotency key if you send one, and charges twice if you do not.

Those are the properties a real integration is actually built against. Everything else a provider does — 3-D Secure, tokenisation, settlement reports — is detail you will read in their documentation. These three are what the *architecture* has to survive, and they are the same for every provider in every country.

14.3 Choosing a Timeout, Which Is Not a Performance Setting

Measured: 200 authorisations, waiting as long as it takes

From `code/m14_gateway.py -latency 200`, against the stack in Appendix A.

Median	321 ms	what a single test request shows you
Mean	700 ms	*more than twice the median*
90th percentile	3,180 ms	
95th percentile	3,774 ms	
99th percentile	4,815 ms	
Slowest answer	4,998 ms	**16× the median**
Never answered	4 of 200	2%

The mean is more than twice the median, which is the whole measurement. A distribution where the mean sits far above the median is one with a long tail, and an average response time of 700 ms describes almost none of the requests: most took a third of a second and a minority took five seconds. Reporting "average response time" here would be worse than reporting nothing, because it sounds like information.

Measured: What each candidate timeout would have cost

The same 200 requests, re-read against each timeout a developer might choose.

Timeout	Answered	Cut off	Share	What the shop is doing
0.5 s	171	25	12.8%	abandoning good payments
1.0 s	173	23	11.7%	abandoning good payments
2.0 s	173	23	11.7%	abandoning good payments
3.0 s	175	21	10.7%	abandoning good payments
5.0 s	**196**	**0**	**0.0%**	waiting
8.0 s	196	0	0.0%	waiting

The counter-intuitive result is the flat middle. Raising the timeout from 1 second to 3 seconds — tripling how long the customer waits — rescues two payments in two hundred. One percent. Almost nothing.

The reason is that the distribution is not a bell curve but *two groups*: requests that answer in under half a second, and requests that take between two and five seconds. There is hardly anything in between, so a timeout placed in between catches hardly anything. **Either you wait for the slow group or you do not, and every value between those two decisions buys you nothing while costing the customer time.**

A timeout chosen from the median rejects one payment in eight. And each rejection is not a failed payment — it is a payment whose outcome the shop does not know.

What to do about the third branch

A timed-out request has an unknown outcome. The shop must choose, in advance, what it believes.

1. **Never assume failure and never assume success.** Record it as *unknown*, which is a third state and needs a place to live — Chapter 10's status 0 is where it goes.

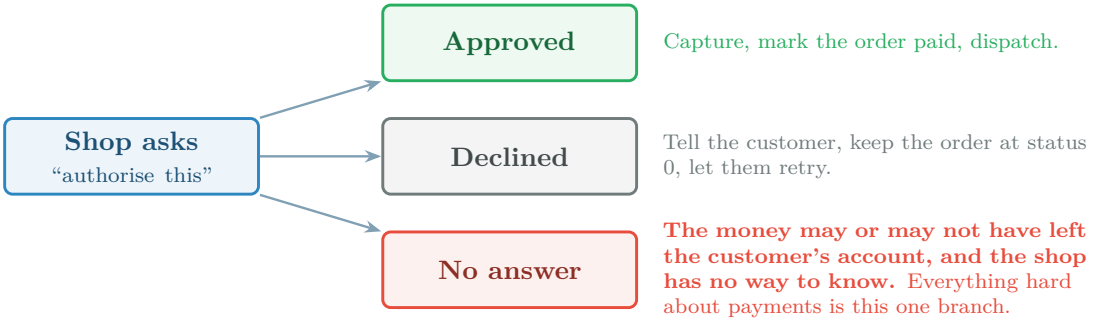


Figure 14.1: The three outcomes of a payment request. Most integration tutorials cover the first two.

2. **Ask the provider.** Every gateway offers a status query. Retry it on a schedule, not in the customer's request.
3. **Make the customer's retry safe.** If they press the button again, the same idempotency key must go with it, or you have charged them twice for being impatient.
4. **Reconcile daily.** The provider's settlement report against your orders. This is the only mechanism that catches everything the other three miss, and it is the one small shops skip.

14.4 Idempotency, Demonstrated

Measured: The same payment, sent twice

From `code/m14_gateway.py -idempotency`.

Without a key:

first	transaction `f59594f7d23c4446`	approved	125 ms
second	transaction `8171b6e85f9b4766`	approved	2,644 ms

Two transactions. The customer paid twice, and both payments are valid — there is nothing for the provider to detect, because two requests to charge the same amount are exactly what a customer buying two of something looks like.

With a key:

first	transaction `50f018abfd9243ec`	approved	149 ms
second	transaction `50f018abfd9243ec`	**replayed**	**5 ms**

The second request returned *the first request's answer*, in 5 ms, without charging anything. The provider saw four requests and charged three times.

Where the key must come from. Not from the request — a retry is a new request and would generate a new key, which defeats the entire mechanism. It must come from *the order*: generated once when the order row is created, stored on it, and sent unchanged with every attempt to pay for that order. This is the second reason Chapter 10's order row exists before the payment, and the better one.

Common Pitfalls

- **Choosing a timeout from the median response time.** It rejected one payment in eight here.
- **Reporting a gateway's "average" latency.** The mean was twice the median; the average describes no request that actually happened.
- **Treating a timeout as a failure** and letting the customer reorder.
- **Generating the idempotency key in the request** rather than storing it on the order.
- **Retrying inside the customer's request.** They are waiting; the retry belongs in a background job.
- **Logging the full request body** while integrating. If the body ever contains a card number, your log is now in PCI scope, and logs are backed up, shipped and searched by people who are not you.
- **Trusting a callback because it arrived.** Verify its signature; anyone can post to a public URL.
- **Skipping daily reconciliation** because the integration works. The integration working is what reconciliation is for.

Try It Yourself: Integrate against a gateway that misbehaves

```
# --- 1. Start the provider and look at it ----------------------------
cd code
docker compose up -d gateway
curl -s http://localhost:8092/health

# --- 2. Read it. It is one file --------------------------------------
# gateway/provider.py. Find the three properties it models, and the
# environment variable that controls each.

# --- 3. Measure before you decide ------------------------------------
python m14_gateway.py --latency 200

# --- 4. Predict your timeout, then check it --------------------------
# Write down the timeout you would have chosen from the median ALONE.
python m14_gateway.py --timeout

# --- 5. Demonstrate the double charge --------------------------------
python m14_gateway.py --idempotency

# --- 6. Make the provider worse, and re-decide -----------------------
# In docker-compose.yml set GATEWAY_TAIL_RATE to 0.35 and
# GATEWAY_HANG_RATE to 0.10, then:
docker compose up -d gateway
python m14_gateway.py --latency 100
# Does your timeout still hold? What changed, the median or the tail?

# --- 7. Put it back --------------------------------------------------
# Restore the defaults and restart the gateway.

# --- 8. Write the integration policy ---------------------------------
# One page, four headings: timeout and why; what an unknown outcome
# means for the order; where the idempotency key comes from; how
# reconciliation happens and who does it.
```

What to notice. Step 6 is the point. Most people's timeout is chosen once and never revisited, and a provider's tail is exactly the thing that changes without telling you — at the end of the month, on sale days, whenever their own upstream is busy.

Checkpoint — Test Yourself

1. A gateway's median response is 321 ms and its 95th percentile is 3,774 ms. A colleague proposes a 1-second timeout, reasoning that it is "three times the median, which is generous". Using the measured table, say what that choice actually costs and what you would propose instead, with your reason.
2. Explain why the measured timeout table barely improves between 1 second and 3 seconds, and what that shape tells you about the gateway that a single average could not.
3. Your shop charges a customer, the request times out, and the customer presses Pay again. Describe exactly what happens with and without an idempotency key, say where the key must be stored, and explain why generating it when the button is pressed would not work.

Chapter Summary

- A hosted page, a direct post and a server-to-server integration are the three architectures, and they map onto SAQ-A, SAQ-A-EP and SAQ-D. The design decision and the compliance decision are the same decision.
- A gateway is three hard properties: it is slow and irregular, it sometimes does not answer, and it tells you twice. The course ships a provider that does all three on purpose.
- Measured: median 321 ms, mean 700 ms, p95 3,774 ms, slowest 4,998 ms — sixteen times the median — and 2% that never answered at all. A mean twice the median is a tail, and an average is the wrong statistic for it.
- Measured: a 1-second timeout cuts off 11.7% of payments and a 3-second timeout cuts off 10.7%. The distribution is two groups with a gap between them, so a timeout in the gap buys nothing.
- A timed-out payment is not a failed payment. It is an unknown outcome, it needs a state of its own, and it is resolved by querying the provider and by daily reconciliation — not by guessing.
- Measured: without an idempotency key the same payment charged twice, with two valid transaction identifiers. With one, the second request returned the first answer in 5 ms and charged nothing.
- The key belongs to the order, not to the request. This is the better reason why the order row exists before the payment.

Review Questions

1. (MCQ) A shop using a hosted payment page completes which self-assessment questionnaire? (a) SAQ-A; (b) SAQ-A-EP; (c) SAQ-D; (d) none is required.

2. (MCQ) In the measured distribution, the mean was 700 ms and the median 321 ms. This indicates: (a) a measurement error; (b) a long tail of slow requests; (c) that half the requests failed; (d) that the gateway is overloaded.

3. (Short) Explain why a timed-out payment request has an unknown outcome rather than a failed one, and give the consequence of assuming either.

4. (Short) Give two things that would move a shop from SAQ-A to a heavier questionnaire, each of which a developer might do without realising.

5. (Short) Why must a callback's signature be verified, given that the callback comes from the provider?

6. (Applied) You are integrating a provider whose published median is 250 ms. Describe the measurement you would run before choosing a timeout, how many samples you would need and

why, which statistics you would report, and what you would do differently if the distribution turned out to be a smooth bell curve instead of two groups.

7. **(Applied)** Design the reconciliation process for a shop taking 200 orders a day. State what two records you compare, how you match them, the three kinds of mismatch you expect to find, what each one means, and who in the business acts on it.